

The Complete Stack Wrapper

Combining It All

A modern production app combines Tailwind for styling, Next.js for routing/API, and components like Shadcn UI (which uses Tailwind under the hood).

Global wrapper: Set text defaults in body.

Dark mode: Use next-themes mapped to Tailwind's dark:.

Container: Use max-w-7xl mx-auto for page wrappers.

```
// app/layout.tsx (The Ultimate Wrapper)
import { Inter } from 'next/font/google';
import { ThemeProvider } from
'@components/theme-provider';
import './globals.css';
const inter = Inter({ subsets: ['latin']
});
export default function RootLayout({
children }) {
  return (
    <html lang="en"
suppressHydrationWarning
    {/* antialiased prevents text
rendering hydration mismatches */}
    <body className={` ${inter.className}
antialiased min-h-screen bg-white dark:bg-
slate-950 text-slate-900 dark:text-slate-
50`} >
      <ThemeProvider attribute="class"
defaultTheme="system" enableSystem>
        <Navbar />
        <main className="flex-1 flex
flex-col">
          {children}
        </main>
        <Footer />
      </ThemeProvider>
    </body>
  </html>
  );
}
```

Video Streaming Interface

Streaming apps require a pure black background (bg-black or bg-[#141414]), absolute positioning for hero overlays, and horizontal scrolling for movie rows.

```
<!-- Hero Movie Overlay -->
<div class="relative h-[80vh] w-full">
  
  <!-- Gradient fade to bottom -->
  <div class="absolute inset-0 bg-
gradient-to-t from-[#141414] via-
transparent to-black/60"></div>
  <!-- Content -->
  <div class="absolute bottom-[20%]
left-10">
    <h1 class="text-6xl font-black
text-white">Movie Title</h1>
    <div class="flex gap-3 mt-1">
      <button class="bg-white text-
black px-6 py-2 rounded font-bold">▶
Play</button>
      <button class="bg-gray-500/70
text-white px-6 py-2 rounded font-
bold">More Info</button>
    </div>
  </div>
</div>
```

```
<!-- Horizontally Scrolling Row -->
<div class="px-10 py-4 -mt-8 relative
z-10">
  <h2 class="text-xl font-bold text-
white mb-4">Trending Now</h2>
  <div class="flex overflow-x-auto
snap-x gap-2 pb-4 hide-scrollbar">
    <!-- Movie Thumbnail -->
    <div class="w-48 shrink-0 snap-
start transition duration-300
hover:scale-105 hover:z-20 cursor-
pointer">
      
    </div>
    <div class="w-48 shrink-0 snap-
start">
      
    </div>
  </div>
</div>
```

The 3-Column Social Layout (Twitter/X Style)

Container Structure

A centered container divided into 3 columns: Left Navigation, Center Feed, and Right Trending.

Wrapper: max-w-7xl mx-auto flex justify-center

Left/Right: w-64 hidden lg:block sticky top-0 h-screen

Center: max-w-2xl w-full border-x border-gray-200 min-h-screen

```
<!-- Feed Post Component -->
<article class="p-4 border-b border-gray-200
hover:bg-gray-50 transition cursor-pointer">
  <div class="flex space-x-3">
    
    <div class="flex-1 min-w-0">
      <div class="flex items-center space-x-1">
        <span class="font-bold text-gray-900
truncate">John Doe</span>
        <span class="text-gray-500 text-sm">@johndoe &middot; 2h</span>
      </div>
      <p class="text-gray-800 mt-1">Just setting
up my twttr. Tailwind makes this so easy!</p>
      <!-- Action Icons (Reply, Retweet, Like) -
->
      <div class="flex justify-between text-
gray-500 mt-3 max-w-md">
        <button class="hover:text-blue-500">💬
      </button>
        <button class="hover:text-green-
500">🔄</button>
        <button class="hover:text-pink-500">❤️
      </button>
      </div>
    </div>
  </div>
</article>
```

Login & Registration View

Perfect Centering

The most common auth layout is a small white card perfectly centered on a light gray background.

Wrapper: `min-h-screen flex items-center justify-center bg-gray-50`

Card: `max-w-md w-full p-8 bg-white rounded-xl shadow-lg`

Inputs: Stacked vertically using `space-y-6` on the form.

```
<div class="min-h-screen flex items-center
justify-center bg-slate-50 px-4">
  <div class="max-w-md w-full bg-white
rounded-2xl shadow-xl p-8 border border-
slate-100">
    <div class="text-center mb-8">
      <h2 class="text-2xl font-bold text-
slate-900">Welcome back</h2>
      <p class="text-sm text-slate-500 mt-
1">Please sign in to your account</p>
    </div>

    <form class="space-y-5">
      <!-- Inputs -->
      <div><label class="block text-sm
font-medium mb-1">Email</label><input
class="..." /></div>
      <div><label class="block text-sm
font-medium mb-1">Password</label><input
class="..." /></div>
      <button class="w-full bg-indigo-600
text-white py-2.5 rounded-lg font-medium
hover:bg-indigo-700 transition">Sign
In</button>
    </form>
  </div>
</div>
```

The Article Layout (@tailwindcss/typography)

Rich Text Rendering

Never try to style markdown or CMS content manually. Always install the Official Typography plugin and use the prose class.

```
$ npm install -D  
@tailwindcss/typography
```

Wrap your dynamic article content in an `article` tag with the `prose` classes to magically style all `h1`, `p`, `a`, `ul`, and `blockquote` tags.

```
<main class="pt-24 pb-16 bg-white">  
  <div class="max-w-3xl mx-auto px-4">  
    <header class="mb-10 text-center">  
      <h1 class="text-4xl font-extrabold text-slate-900 mb-4">Blog Post Title</h1>  
      <p class="text-slate-500">Published on Oct 24, 2023</p>  
    </header>  
  
    <!-- Magic happens here -->  
    <article class="prose prose-lg prose-blue mx-auto dark:prose-invert">  
      <!-- Injected HTML from Markdown/CMS goes here -->  
      <p>This is a paragraph...</p>  
      <h2>Subheading</h2>  
    </article>  
  </div>  
</main>
```

Landing Page Structure

A modern SaaS landing page usually consists of stacked, full-width sections with centered content containers.

Navbar: fixed w-full top-0 bg-white/80 backdrop-blur-md

Hero: pt-32 pb-20 text-center max-w-4xl mx-auto

Features Grid: grid md:grid-cols-3 gap-8 py-24

CTA Section: bg-slate-900 text-white py-20

```
<section class="py-24 bg-gray-50">
  <div class="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
    <div class="text-center mb-16">
      <h2 class="text-3xl font-bold">Features</h2>
    </div>
    <div class="grid grid-cols-1 md:grid-cols-3 gap-8">
      <!-- Feature Cards -->
    </div>
  </div>
</section>
```

Portfolio Website

Portfolios rely heavily on masonry grids and distinct typography.

Hero Intro: text-left max-w-2xl py-32

Project Grid: grid grid-cols-2 md:grid-cols-3 gap-4

Project Card: group relative overflow-hidden rounded-xl

```
<!-- Project Card with Hover Effect -->
<div class="group relative aspect-square bg-gray-200 overflow-hidden rounded-2xl">
  
  <div class="absolute inset-0 bg-gradient-to-t from-black/80 via-transparent opacity-0 group-hover:opacity-100 transition-opacity flex items-end p-6">
    <h3 class="text-white font-bold">Project Name</h3>
  </div>
</div>
```

Screen Reader Utilities

Visually hide elements while keeping them available for screen readers (Crucial for icon-only buttons).

```
<button class="p-2 bg-gray-100
rounded-full">
  <svg>...</svg>
  <!-- Hidden visually, read by
screen readers -->
  <span class="sr-only">Close
Modal</span>
</button>

<!-- Undo sr-only on focus -->
<a href="#content" class="sr-only
focus:not-sr-only">
  Skip to content
</a>
```

Focus & ARIA States

Focus Visible

Use `focus-visible` instead of `focus` to only show focus rings when navigating via keyboard, avoiding ugly outlines on mouse clicks.

ARIA Variants

Style elements based on their ARIA attributes.

```
aria-checked:bg-blue-500
aria-expanded:rotate-180
```

Keeping Your Codebase Clean

1. Never Construct Class Names

As mentioned in the Performance chapter, Tailwind's PurgeCSS engine scans for unbroken strings. If you split strings, the class gets purged.

```
// ❌ Fails (Purged)
<div class=bg-red-500>
```

2. Avoid Magic Numbers

Try to avoid overusing Arbitrary Values (like `w-[23px]`). If a value is used multiple times, add it to your `tailwind.config.js` to keep your design system consistent.

3. Managing Class Clutter

Long class strings are the #1 complaint about Tailwind. Mitigate this by:

1. Formatting your file using the **Prettier plugin** to auto-wrap long class strings logically.
2. Using tools like `clsx` or `classnames` to split complex conditional logic onto multiple lines.
3. Extracting UI structures (Buttons, Inputs, Cards) into framework components immediately. The actual file using the button shouldn't have any Tailwind styling on it.

```
<Button variant="danger" size="lg">Delete</Button>
```

The Recommended Class Order

To keep teams consistent, Tailwind recommends ordering classes in a specific semantic way, starting from the outside of the box model to the inside.

1. **Layout:** container, display, position (absolute, flex, grid)
2. **Dimensions:** width, height, min-w, max-h
3. **Spacing:** margin, padding
4. **Typography:** text-size, font, leading, tracking
5. **Backgrounds & Borders:** bg, border, rounded
6. **Effects:** shadow, opacity, mix-blend
7. **Interactivity & States:** hover:, focus:, sm:, dark:

Automatic Sorting

Don't sort classes manually! Use the official Prettier plugin to automatically organize your classes on save.

```
$ npm install -D prettier \
$ prettier-plugin-tailwindcss
```

This guarantees every developer on your team writes classes in the exact same order.

When to Extract Components (And When Not To)

The Golden Rule

Only extract components when you are dealing with **structural duplication**, not just visual duplication.

Bad Practice: Using `@apply` in CSS to create a `.btn` class. This breaks the utility-first workflow and leads to bloated CSS files again.

Good Practice: Extracting a reusable React, Vue, or Blade component (e.g., `<Button />`) that contains the Tailwind classes.

```
// ✅ Correct: Extracting to a React Component
function Button({ children, variant = 'primary' }) {
  const baseStyle = "px-4 py-2 font-bold rounded-lg transition-colors";
  const variants = {
    primary: "bg-blue-600 text-white",
    hover: "bg-blue-700",
    outline: "border-2 border-blue-600 text-blue-600 hover:bg-blue-50"
  };

  return (
    <button className={`${baseStyle} ${variants[variant]}`}>
      {children}
    </button>
  );
}
```

Dropdown Menu (CSS Only)

Uses the `group` variant to show the menu on hover. (In React, you'd use state instead).

```
<!-- Wrapper with 'group' -->
<div class="relative inline-block group">
  <button class="px-4 py-2 bg-white border
rounded-lg shadow-sm">
    Options &darr;
  </button>

  <!-- Menu (Hidden by default, shown on
group-hover) -->
  <div class="absolute right-0 mt-1 w-48
bg-white border rounded-lg shadow-xl
hidden group-hover:block z-50">
    <div class="py-1">
      <a href="#" class="block px-4 py-2
hover:bg-gray-100">Edit Profile</a>
      <a href="#" class="block px-4 py-2
hover:bg-gray-100 text-red-600">Logout</a>
    </div>
  </div>
</div>
```

Toast Notification

Fixed to a corner of the screen using absolute/fixed positioning.

```
<!-- Fixed to Bottom-Right -->
<div class="fixed bottom-4 right-
4 z-50 flex items-center w-full
max-w-xs p-4 space-x-3 text-gray-
800 bg-white border rounded-lg
shadow-xl">
  <div class="inline-flex items-
center justify-center shrink-0 w-
8 h-8 text-green-500 bg-green-100
rounded-lg">
    ✓
  </div>
  <div class="text-sm font-
medium">Item saved successfully.
</div>
  <button class="ml-auto text-
gray-400 hover:text-gray-
900">&times;</button>
</div>
```

Modal / Dialog Window

Modal Architecture

A modal requires a fixed, full-screen overlay (backdrop) that centers the actual dialog box inside it.

Backdrop: `fixed inset-0 z-50 bg-black/50 backdrop-blur-sm`

Centering: `flex items-center justify-center` on the backdrop.

Dialog: `bg-white rounded-xl shadow-2xl p-6`

```

<!-- Modal Overlay (Backdrop) -->
<div class="fixed inset-0 z-50 flex items-center
justify-center bg-black/50 backdrop-blur-sm p-
4">

  <!-- Modal Dialog Box -->
  <div class="bg-white rounded-2xl shadow-2xl w-
full max-w-md overflow-hidden">
    <div class="p-6 border-b border-gray-100
flex justify-between items-center">
      <h3 class="text-lg font-bold">Confirm
Deletion</h3>
      <button class="text-gray-400 hover:text-
gray-600">&times;</button>
    </div>
    <div class="p-6 text-gray-600">
      Are you sure you want to delete this item?
      This action cannot be undone.
    </div>
    <div class="p-4 bg-gray-50 flex justify-end
gap-3">
      <button class="px-4 py-2 bg-white border
rounded-lg">Cancel</button>
      <button class="px-4 py-2 bg-red-600 text-
white rounded-lg">Delete</button>
    </div>
  </div>
</div>

```

Top Navbar

Uses flex and justify-between to space the logo and the menu.

```
<nav class="bg-white border-b border-gray-200 px-6 py-4 flex items-center justify-between sticky top-0 z-50">
  <!-- Brand Logo -->
  <a href="#" class="text-xl font-bold text-blue-600">
    BrandLogo
  </a>

  <!-- Desktop Menu -->
  <div class="hidden md:flex space-x-8 items-center">
    <a href="#" class="text-gray-600 hover:text-blue-600">Home</a>
    <a href="#" class="text-gray-600 hover:text-blue-600">About</a>
    <button class="px-4 py-2 bg-blue-600 text-white rounded-lg">Sign In</button>
  </div>
</nav>
```

Sidebar Navigation

A vertical flex container pinned to the left side.

```
<aside class="w-64 h-screen bg-gray-900 text-white flex flex-col p-4">
  <div class="text-2xl font-bold mb-8 px-2">Admin Panel</div>

  <nav class="flex-1 flex flex-col space-y-2">
    <a href="#" class="px-4 py-2.5 bg-gray-800 rounded-lg text-blue-400 font-medium">Dashboard</a>
    <a href="#" class="px-4 py-2.5 hover:bg-gray-800 rounded-lg transition-colors">Users</a>
    <a href="#" class="px-4 py-2.5 hover:bg-gray-800 rounded-lg transition-colors">Settings</a>
  </nav>
</aside>
```

Buttons

Primary & Outline

Combine padding, background, text, and hover states.

```
<!-- Primary Button -->
<button class="px-5 py-2.5 bg-blue-600 text-white font-medium rounded-lg hover:bg-blue-700 focus:ring-4 focus:ring-blue-300 transition-colors">
  Save Changes
</button>

<!-- Outline Button -->
<button class="px-5 py-2.5 border-2 border-gray-300 text-gray-700 font-medium rounded-lg hover:bg-gray-50 transition-colors">
  Cancel
</button>
```

Cards & Profiles

A classic card layout with an image header and content body.

```
<div class="max-w-sm bg-white border border-gray-200 rounded-2xl shadow-md overflow-hidden">
  <!-- Card Image -->
  

  <!-- Card Body -->
  <div class="p-5">
    <h5 class="mb-2 text-xl font-bold tracking-tight text-gray-900">Card Title</h5>
    <p class="mb-4 text-gray-600">Here are the biggest enterprise technology acquisitions of 2021.</p>
    <a href="#" class="inline-flex items-center px-4 py-2 bg-blue-600 text-white rounded-lg hover:bg-blue-700">
      Read more &rarr;
    </a>
  </div>
</div>
```

React Fundamentals

className vs class

In React, always use `className` instead of `class`.

Dynamic Classes (clsx & twMerge)

To safely merge Tailwind classes conditionally without style conflicts, the community standard is combining `clsx` and `tailwind-merge`.

```
import { clsx } from 'clsx';
import { twMerge } from 'tailwind-merge';
// Standard utility function (often named 'cn')
export function cn(...inputs) {
  return twMerge(clsx(inputs));
}
// Usage in component
<button className={cn(
  "px-4 py-2 bg-blue-500",
  isError && "bg-red-500"
)}>
```

Next.js Patterns

Server Components

Tailwind works perfectly with React Server Components (RSC) in Next.js App Router because CSS generation happens at build time.

Global CSS Location

Import your `globals.css` (containing Tailwind directives) directly into your root `layout.tsx`.

```
// app/layout.tsx
import './globals.css';
export default function
RootLayout({ children }) {
  return (
    <html lang="en">
      <body className="bg-slate-
50">{children}</body>
    </html>
  )
}
```

Tree Shaking & Content Discovery

Tailwind doesn't ship a giant CSS file. It **scans your source files** (HTML, JS, JSX) for class names and generates *only* the CSS you actually used.

Content Configuration

The content array in your config tells Tailwind exactly where to look for class names.

Tree Shaking

Anything not found in your files is automatically stripped out, keeping the CSS bundle size constant.

```
// tailwind.config.js
module.exports = {
  content: [
    './src/**/*.html',
    './src/**/*.js',
    './src/**/*.jsx',
    './src/**/*.ts',
    './src/**/*.tsx',
  ],
  // ...
}
```

CSS Optimization

Tailwind automatically applies optimizations during the build process to minimize your output.

- **Minification:** Removes all unnecessary whitespace, comments, and newlines.
- **Grouping:** Collapses identical CSS rules into single declarations.
- **Prefixing:** Automatically adds vendor prefixes (Autoprefixer) so styles work on all browsers.

Strategies for Tiny Bundle Sizes

Production Workflow

When running `npm run build`, Tailwind automatically sets `NODE_ENV=production`, which triggers:

1. Purging unused CSS.
2. Minifying the CSS.
3. Compression (Gzip/Brotli).

Avoid Dynamic Classes

Tailwind cannot "see" dynamic string interpolation. Avoid building class names dynamically because the purge engine will miss them.

Don't: `class={`text-${err ? 'red' : 'green'}-500`}`
Do: `class={err ? 'text-red-500' : 'text-green-500'}`

The @theme Directive (v4)

Tailwind CSS v4 moves away from `tailwind.config.js` to a CSS-first configuration model. You now define your entire theme directly in CSS using the `@theme` directive alongside CSS variables.

```
@import "tailwindcss";
/* Define your custom theme completely in CSS */
@theme {
  --color-brand-500: #0ea5e9;
  --color-brand-600: #0284c7;

  --font-display: "Oswald", sans-serif;

  --breakpoint-3xl: 1920px;
}
```

Adding `--color-brand-500` automatically generates utilities like `text-brand-500` and `bg-brand-500`.

@utility and @variant Directives

@utility (Custom Classes)

In v4, replace `@layer` utilities with the much cleaner `@utility` block. It automatically supports hover states and breakpoints.

```
@import "tailwindcss";
@utility content-auto {
  content-visibility: auto;
}
@utility neon-text {
  text-shadow: 0 0 10px theme(--color-cyan-500);
}
```

@variant (Custom States)

Create custom state modifiers (like `hover:` or `focus:`) directly via CSS instead of needing complex JavaScript plugins.

```
@import "tailwindcss";
/* Create a 'pointer-coarse:' variant for touch devices */
@variant pointer-coarse {
  @media (pointer: coarse) {
    @slot;
  }
}
/* Create an 'nth-3:' variant */
@variant nth-3 {
  &:nth-child(3) {
    @slot;
  }
}
```

@import & @layer (Core Setup)

Initial Directives (@import)

Tailwind provides custom CSS directives to inject its features. In Tailwind v3, we used `@tailwind`. In **Tailwind v4**, we use standard CSS `@import`.

```
/* Tailwind v3 (Legacy) */
@tailwind base;
@tailwind components;
@tailwind utilities;
/* Tailwind v4 (Modern) */
@import "tailwindcss";
@import "tailwindcss/utilities";
```

Organizing CSS with @layer

Tells Tailwind which "bucket" your custom CSS belongs to. This ensures your custom styles are compiled in the right order (Base → Components → Utilities) and can be purged properly.

```
@layer base {
  h1 { @apply text-4xl font-bold; }
}
@layer components {
  .btn { @apply px-4 py-2 bg-blue-500 rounded; }
}
@layer utilities {
  .content-auto { content-visibility: auto; }
}
```

Component Extraction (@apply & @config)

@apply and @config Directives

Avoid Dynamic Classes

Tailwind cannot "see" dynamic string interpolation. Avoid building class names dynamically because the purge engine will miss them.

```
Don't: class={`text-${err ? "red" : "green"}-500`}
Do: class={err ? "text-red-500" : "text-green-500"}
```

Custom Sizes and Custom Breakpoints

Spacing

Adding to spacing automatically creates utilities for padding, margin, width, height, maxHeight, gap, inset, space, and translate.

Screens (Breakpoints)

You can add larger breakpoints (like 3xl) or completely replace the default breakpoints with your own names (like tablet, desktop).

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      spacing: {
        '128': '32rem', // Creates w-128,
        // mt-128, etc.
        '144': '36rem',
      },
      screens: {
        '3xl': '1600px', // Adds a new
        // larger breakpoint
      }
    }
  }
}
```

Official Plugins

Tailwind provides official plugins for complex styles.

- **@tailwindcss/typography:** Adds the prose class for beautiful rich-text rendering.
- **@tailwindcss/forms:** Resets and nicely styles native form elements.

```
plugins: [
  require('@tailwindcss/typography'),
  require('@tailwindcss/forms'),
],
```

Adding Custom Utilities

Use the `@layer` utilities directive in your main CSS file to add custom classes that still support Tailwind modifiers (like `hover:` or `md:`).

```
/* src/index.css */
@tailwind base;
@tailwind components;
@tailwind utilities;
@layer utilities {
  .text-shadow {
    text-shadow: 2px 2px 4px
    rgba(0,0,0,0.5);
  }
}
```

Extend vs Override

Everything in Tailwind can be customized via `tailwind.config.js`. The most important concept is the difference between replacing the theme and extending it.

- **Inside theme: { ... }**
Completely **OVERWRITES** the default Tailwind utilities.
- **Inside theme: { extend: { ... } }**
KEEPS default utilities and adds your new ones.
(Always recommended).

tailwind.config.js Skeleton

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ["/src/**/*.{js,jsx,ts,tsx}"],
  theme: {
    // OVERRIDE default theme here
    extend: {
      // ADD to default theme here
    },
  },
  plugins: [],
}
```

Adding Brand Colors & Custom Fonts

Adding Colors

Once added to colors, Tailwind automatically generates all related classes (`bg-brand`, `text-brand`, `border-brand`, `ring-brand`, etc).

Adding Fonts

Usually requires importing the font via CSS or HTML, then defining the family in the config.

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        brand: '#00D1FF',
        primary: {
          100: '#E6FAFF',
          500: '#00B8E6',
          900: '#005C73'
        }
      },
    },
    fontFamily: {
      sans: ['Inter', 'sans-serif'], //
      // Overrides default sans
      display: ['Poppins', 'sans-serif'], //
      // Adds font-display
    }
  }
}
```

The Mobile-First Approach

Tailwind uses a **mobile-first** breakpoint system. This means unprefixed utilities (like `text-center`) take effect on all screen sizes, while prefixed utilities (like `md:text-left`) only take effect at that breakpoint **and above**.

Rule of Thumb: Design your layout for mobile phones first without any prefixes, then use `sm:`, `md:`, or `lg:` to adjust the layout for larger screens.

Default Breakpoints Reference

Prefix Minimum Width CSS Equivalent

sm	640px	@media (min-width: 640px)
md	768px	@media (min-width: 768px)
lg	1024px	@media (min-width: 1024px)
xl	1280px	@media (min-width: 1280px)
2xl	1536px	@media (min-width: 1536px)

Practical Responsive Layout Patterns

1. Responsive Grids

Easily switch from stacked layouts on mobile to multi-column grids on desktop.

```
<!-- Mobile: 1 col, Tablet: 2 cols, Desktop: 4 cols -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
  <div class="bg-blue-500">Box 1</div>
  <div class="bg-blue-500">Box 2</div>
  <div class="bg-blue-500">Box 3</div>
  <div class="bg-blue-500">Box 4</div>
</div>
```

2. Show / Hide Elements

Perfect for mobile menus, hamburger icons, and hiding sidebars on small screens.

```
<!-- Hamburger icon visible ONLY on mobile -->
<button class="block md:hidden">
  <svg>Hamburger Icon</svg>
</button>
<!-- Navigation links visible ONLY on tablet & up -->
<nav class="hidden md:flex space-x-6">
  <a href="#">Home</a>
  <a href="#">About</a>
</nav>
```

Scroll Behavior

Apply `scroll-smooth` to your `<html>` tag to globally enable smooth scrolling when a user clicks an anchor link (e.g., `<a href="#section">`).

```
<!-- Global smooth scrolling -->
<html class="scroll-smooth">
  <body>
    <a href="#about">Go to About</a>
    <!-- ... -->
    <section id="about">About Section</section>
  </body>
</html>
```

Building a Snapping Carousel

Tailwind includes utilities for CSS Scroll Snapping, allowing you to build native, JS-free image carousels and sliders.

This creates a scrollable row of images that automatically lock into the center when the user finishes swiping/scrolling.

1. Container Classes

snap-x: Enables horizontal snapping

snap-mandatory: Forces stopping on an item

snap-proximity: Snaps only if near an item

2. Item Classes

snap-start: Snaps to left edge

snap-center: Snaps item to center

```
<!-- The Container -->
<div class="flex overflow-x-auto snap-x snap-
mandatory gap-4 pb-4">
  <!-- The Items -->
  <div class="snap-center shrink-0 w-80 h-40 bg-
blue-200 rounded-xl">
    Card 1
  </div>
  <div class="snap-center shrink-0 w-80 h-40 bg-
pink-200 rounded-xl">
    Card 2
  </div>
  <div class="snap-center shrink-0 w-80 h-40 bg-
yellow-200 rounded-xl">
    Card 3
  </div>
</div>
```

Cursors & Pointer Events

Cursor Styles

cursor-pointer: Hand icon

cursor-wait: Spinner

cursor-not-allowed: Blocked

cursor-text: Text I-beam

Pointer Events

pointer-events-none: Element ignores mouse clicks (clicks pass through it).

pointer-events-auto: Restores default behavior.

Text Selection & Resize

User Select

select-none: Prevents text highlighting

select-text: Allows selection

select-all: Selects all text on click

Resizing (Textareas)

resize-none: Disables resizing

resize-y: Vertical resizing only

resize-x: Horizontal resizing only

Accent Color & Appearance

Accent Color

Quickly theme standard HTML form controls like checkboxes, radio buttons, and range sliders without writing complex custom CSS.

```
<!-- A pink checkbox! -->
<label>
  <input type="checkbox" class="accent-pink-500 size-4" />
  Subscribe
</label>
```

Appearance None

The appearance-none utility removes native browser styling from elements like `<select>` dropdowns, allowing you to completely custom-style them.

```
<!-- Custom dropdown without default arrow -->
<select class="appearance-none bg-white border border-gray-300 rounded px-4 py-2">
  <option>Option 1</option>
</select>
```

Custom Animations via tailwind.config.js

1. Define in Config

You can add your own custom CSS animations directly into your Tailwind theme.

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      keyframes: {
        wiggle: {
          '0%, 100%': { transform:
'rotate(-3deg)' },
          '50%': { transform:
'rotate(3deg)' },
        },
      },
      animation: {
        'wiggle': 'wiggle 1s ease-in-
out infinite',
      },
    },
  },
}
```

2. Use in HTML

Once defined, Tailwind generates the `animate-wiggle` utility automatically.

```
<div class="animate-wiggle">
  Hover me to wiggle!
</div>
```

Arbitrary Animations (JIT)

For one-off animations, you don't even need the config file:

```
<div class="animate-[wiggle_1s_ease-in-
out_infinite]">
  Look at me go
</div>
```

Transition Properties & Duration

Transition Property

Controls which CSS properties transition.

transition-none: Removes transitions

transition-all: Transitions all properties

transition: Default (color, opacity, shadow, transform)

transition-colors: Only colors

transition-opacity: Only opacity

transition-transform: Only transform

Duration

Controls the length of time the transition takes.

duration-75: 75ms

duration-100: 100ms

duration-150: 150ms
(Default)

duration-200: 200ms

duration-300: 300ms

duration-500: 500ms

duration-700: 700ms

duration-1000: 1000ms

Timing, Delay & Examples

Timing Functions & Delays

Timing Function (Easing)

ease-linear: Constant speed

ease-in: Starts slow, ends fast

ease-out: Starts fast, ends slow

ease-in-out: Slow start and end
(Default)

Delay

Waits before starting the transition.

delay-75, delay-150, delay-300, delay-500...

To create smooth interactions, combine these 4 utilities with a state variant (like `hover:`).

```
<!-- Smooth color & transform on hover -->
<button class="
  bg-cyan-500 hover:bg-cyan-600
  hover:-translate-y-1 hover:shadow-lg
  transition-all duration-300 ease-in-out
">
  Hover Me!
</button>
```

Fill & Stroke Basics

Fill

fill-current: Inherits text color
fill-transparent: Transparent fill
fill-blue-500: Specific color

Stroke

stroke-current: Inherits text color
stroke-none: No stroke
stroke-red-500: Specific color

Stroke Width & Inheritance

Stroke Width

stroke-0: 0px width
stroke-1: 1px width
stroke-2: 2px width

Best Practice: Set **fill-current** or **stroke-current** on the SVG tag, and apply standard text color utilities (e.g., **text-cyan-500**) to the parent element for easy hovering effects.

Practical SVG Example

Sizing and Coloring SVGs

Tailwind makes integrating inline SVGs (like Heroicons) extremely simple.

size-6: Controls both width and height to 1.5rem.

text-purple-600: Passes color to **stroke-current**.

hover:text-purple-800: Easily animates SVG color on hover.

```
<!-- An outlined SVG icon scaled and colored -->
<button class="flex items-center text-purple-600
hover:text-purple-800">
  <svg class="size-6 stroke-current stroke-2
fill-none" viewBox="0 0 24 24">
    <!-- SVG Path -->
    <path d="M12 4v16m8-8H4" />
  </svg>
  <span class="ml-2 font-medium">Add Item</span>
</button>
```

Image & Element Filters

Apply CSS filters directly to elements.

blur-sm / blur-md: Blurs the element	brightness-50 / 150: Adjusts brightness
contrast-75 / 125: Adjusts contrast	grayscale: Makes element B&W
invert: Inverts colors	sepia: Applies sepia tone

Note: You can combine multiple filters. e.g., `blur-sm grayscale`

Drop Shadow vs Box Shadow

Unlike `shadow-*` which applies to the element's bounding box, `drop-shadow-*` follows the alpha channel (exact shape) of an image or SVG.

drop-shadow-sm

drop-shadow-md

drop-shadow-xl

```
<!-- Shadow follows the
transparent PNG shape -->

```

Backdrop Blur & Mix Blend Mode

Backdrop Filters

Applies filters to the area **behind** an element. The element must have a transparent/semi-transparent background.

backdrop-blur-md: Blurs what's behind
backdrop-brightness-50: Darkens backdrop
backdrop-contrast-125: Increases backdrop contrast

Mix Blend Mode

Dictates how an element's content blends with its background.

mix-blend-multiply, **mix-blend-screen**,
mix-blend-overlay

The Glassmorphism Effect:

Combine semi-transparent white backgrounds with backdrop-blur for a frosted glass look.

```
<!-- Frosted Glass Card -->
<div class="bg-white/30 backdrop-blur-md
border border-white/20 shadow-xl rounded-xl
p-6">
  <h3 class="text-black">Glassmorphism</h3>
</div>
```

Ring Utilities

Creates an outline using box-shadow. Doesn't affect layout width.

ring: 3px solid ring (Default)

ring-2 / ring-4: Custom widths

ring-blue-500: Ring color

ring-offset-2: Space between element and ring

ring-inset: Draws ring inside the border



```
<!-- Perfect focus state -->
<button class="focus:outline-none
focus:ring-2 focus:ring-blue-500
focus:ring-offset-2">
  Click me
</button>
```

Outline Utilities

Standard CSS outlines (drawn outside the border edge).

outline: Default 1px outline

outline-none: Removes default outline (Crucial for custom focus states)

outline-dashed: Outline style

outline-offset-2: Outward space

Shadows & Colored Shadows

Box Shadow

Adds beautiful, smooth drop shadows to elements.

shadow-sm: Subtle shadow (good for inputs)

shadow: Default shadow (good for cards)

shadow-md: Medium elevation

shadow-lg: High elevation (modals/dropdowns)

shadow-xl / 2xl: Highest elevations

shadow-inner: Inset shadow

shadow-none: Removes existing shadow

Colored Shadows

You can easily change the color of the shadow to match your brand or create glowing effects.



```
<!-- Standard Shadow -->
<div class="shadow-lg bg-white">...
</div>
<!-- Colored Glowing Button -->
<button class="bg-cyan-500 shadow-lg
shadow-cyan-500/50 text-white rounded-
full px-4 py-2">
  Glow Effect
</button>
```

Width, Style & Color

Width & Sides

border: 1px all sides
border-2: 2px all sides
border-x: Left & Right
border-t-4: Top 4px
border-0: Remove border

Style & Color

border-solid: Default style
border-dashed / dotted
border-blue-500: Solid color
border-black/50: 50% opacity

Border Radius

Rounds the corners of an element.

rounded-sm: 0.125rem
rounded: 0.25rem (Default)
rounded-md: 0.375rem
rounded-lg: 0.5rem
rounded-full: 9999px (Pills/Circles)

Specific sides: rounded-t-lg (top), rounded-bl-full (bottom-left)

Borders Between Children

The **divide-*** utilities add borders **between** child elements. It's the border equivalent of the **space-*** utility, perfect for lists, tables, or flex/grid containers where you only want inner borders.

divide-y: 1px border between stacked children
divide-x-2: 2px border between side-by-side children
divide-gray-200: Sets the divider color
divide-dashed: Sets the divider style

```
<!-- Creates a bordered list without
border on top/bottom -->
<ul class="flex flex-col divide-y
divide-gray-200">
  <li class="py-2">Item 01</li>
  <li class="py-2">Item 02</li>
  <li class="py-2">Item 03</li>
</ul>
```

Colors & Images

Background Color

Prefix with `bg-` followed by color name and shade.

`bg-white` / `bg-black` `bg-transparent`

`bg-blue-500` `bg-slate-900`

Background Image

```
<!-- Using arbitrary values for images -->
<div class="bg-[url('/hero.jpg')]">
  Hero Content
</div>
```

Size, Position & Repeat

Size & Repeat

`bg-cover`: Fill container
`bg-contain`: Fit without crop
`bg-auto`: Default size
`bg-no-repeat`
`bg-repeat-x` / `y`

Position

`bg-center`
`bg-top` / `bg-bottom`
`bg-left` / `bg-right`
`bg-left-top`, etc.

```
<!-- Common full-cover background setup -->
>
<div class="bg-[url('bg.png')] bg-cover
  bg-center bg-no-repeat">
</div>
```

Advanced Background Effects

Linear Gradients

Combine direction, starting color, optional middle color, and ending color.

Direction: `bg-gradient-to-t`, `tr`, `r`, `br`, `b`, `bl`, `l`, `tl`

Colors: `from-{color}`, `via-{color}`, `to-{color}`

```
<!-- Blue to Purple gradient going right -->
<div class="bg-gradient-to-r from-cyan-500 to-purple-500">
  Gradient BG
</div>
<!-- 3-color gradient -->
<div class="bg-gradient-to-br from-pink-500 via-red-500 to-yellow-500">...</div>
```

Opacity & Blending

Control transparency and element blending.

opacity-*: 0 to 100
 (e.g., `opacity-50` = 50%)

bg-opacity-*: Target bg only

mix-blend-*: Blends element with its parent (multiply, screen)

bg-blend-*: Blends bg-image with bg-color

Alignment & Transforms

Alignment

text-left
text-center
text-right
text-justify

Transform

uppercase
lowercase
capitalize
normal-case

Decoration & Style

Style

italic
not-italic

Decoration

underline
line-through
no-underline

Customize underlines: underline decoration-wavy
decoration-red-500 underline-offset-4

Text Overflow, White Space, and Lists

Truncation & Wrapping

truncate: Prevents wrapping, hides overflow, adds ellipsis ('...'). (Must have a set width to work)

text-ellipsis: Adds ellipsis (needs overflow-hidden).

whitespace-nowrap: Forces text onto a single line.

break-words: Adds line breaks to prevent long words from overflowing.

break-all: Breaks at any character to prevent overflow.



```
<!-- Automatically adds '...' if text is too long -->
<div class="w-32 truncate">
  This is a very long sentence that will be cut off.
</div>
```

List Styles

Tailwind removes default list styling. Use these to bring them back.

list-none: No bullets

list-disc: Bullet points

list-decimal: Numbers (1, 2, 3)

list-inside: Bullets inside content flow

list-outside: Bullets outside flow (Default)



```
<ul class="list-disc list-inside">
  <li>First Item</li>
  <li>Second Item</li>
</ul>
```

Font Family & Weight

Family

font-sans: Default UI font
font-serif: Serif font
font-mono: Monospaced

Weight

font-light: 300
font-normal: 400 (Default)
font-semibold: 600
font-bold: 700
font-black: 900

Text Color

Controls the text color using the theme palette.

text-black / text-white
text-slate-900: Dark gray
text-blue-500: Primary blue
text-transparent: Transparent

```
<!-- Gradient Text -->
<h1 class="text-transparent bg-clip-text bg-gradient-to-r from-cyan-500 to-blue-500">
  Gradient
</h1>
```

Font Size, Leading, and Letter Spacing

Font Size (text-*)

Tailwind scales font-size and line-height together optimally.

text-xs: 0.75rem	text-sm: 0.875rem
text-base: 1rem (Default)	text-lg: 1.125rem
text-xl: 1.25rem	text-2xl: 1.5rem
text-4xl: 2.25rem	text-6xl: 3.75rem

Line Height (leading-*)

leading-none: 1 (tightest)
leading-tight: 1.25
leading-normal: 1.5 (Standard body text)
leading-loose: 2 (widest)

Letter Spacing (tracking-*)

tracking-tighter: -0.05em
tracking-normal: 0em
tracking-widest: 0.1em (Great for uppercase)

Initialize Grid

Use grid to create a grid container.

```
grid: display: grid;
```

```
inline-grid: display: inline-grid;
```

Grid is perfect for 2D layouts (rows AND columns simultaneously), whereas Flexbox is best for 1D (a row OR a column).

Columns & Rows

Columns

grid-cols-1 to 12

grid-cols-none: No grid cols

Rows

grid-rows-1 to 6

grid-rows-none: No grid rows

```

<!-- 1 col on mobile, 3 cols on desktop -->
<div class="grid grid-cols-1 md:grid-cols-3 gap-4">
  <div>Item 1</div> <div>Item 2</div> <div>Item
3</div>
</div>

```

Col Span, Row Span, Start & End

Column Spanning

col-span-2: Span 2 columns

col-span-full: Span all available columns

col-start-2: Start at grid line 2

col-end-5: End at grid line 5

Row Spanning

row-span-3: Span 3 rows

row-span-full: Span all available rows

row-start-2 / row-end-4: Start/End lines

Create complex bento-box style layouts easily by combining col-span and row-span.

```

<div class="grid grid-cols-3 gap-4">
  <!-- Spans 2 cols, standard row -->
  <div class="col-span-2 bg-blue-200">A</div>

  <!-- Spans 2 rows downwards -->
  <div class="row-span-2 bg-green-200">B</div>

  <!-- Standard 1x1 block -->
  <div class="bg-yellow-200">C</div>

  <!-- Explicitly start at col 2 -->
  <div class="col-start-2 bg-red-200">D</div>
</div>

```

Direction & Wrap

flex-row: Default (Left to Right)
flex-col: Top to Bottom

flex-row-reverse: Right to Left
flex-col-reverse: Bottom to Top

flex-wrap: Wrap items onto multiple lines
flex-nowrap: Prevent wrapping (default)

```
<!-- A vertical flex column with wrapping -->
<div class="flex flex-col flex-wrap h-64">
  <div>1</div><div>2</div><div>3</div>
</div>
```

Gap (Spacing)

The gap utility handles spacing between flex and grid items automatically, replacing complex margins.

gap-4: 1rem spacing on all sides
gap-x-4: 1rem horizontal spacing
gap-y-4: 1rem vertical spacing

```
<div class="flex gap-4">
  <div class="w-10">A</div>
  <div class="w-10">B</div>
</div>
```

Flex Grow, Shrink, and Basis

Control how individual items grow or shrink to fill the flex container.

flex-1: Item can grow/shrink. Ignores initial size. (flex: 1 1 0%)

flex-auto: Item can grow/shrink. Respects initial size.

flex-initial: Can shrink, but won't grow.

flex-none: Prevents growing and shrinking.

Grow & Shrink

grow: Allow to grow
grow-0: Prevent growing
shrink: Allow to shrink
shrink-0: Prevent shrinking

Flex Basis

basis-full: Initial size 100%
basis-1/2: Initial size 50%
basis-0: Initial size 0px

```
<!-- Middle item grows to take remaining space -->
<div class="flex">
  <div class="flex-none w-16">Left</div>
  <div class="grow">Fills Space</div>
  <div class="flex-none w-16">Right</div>
</div>
```

Position Types & Insets

`relative` `absolute` `fixed` `sticky`

Use directional utilities to place elements (top, right, bottom, left). `inset` targets multiple sides.

```
<!-- Fixed Navbar -->
<nav class="fixed top-0 inset-x-0 bg-white
z-50">...</nav>

<!-- Absolute Centering -->
<div class="relative w-32 h-32">
  <div class="absolute top-1/2 left-1/2 -
translate-x-1/2 -translate-y-1/2">
    Center
  </div>
</div>
```

Z-Index & Isolation

Controls the stack order of elements. Values: `z-0`, `z-10`, `z-20` ... up to `z-50`.

Use negative z-index to place behind: `-z-10`.

Isolation:

Use `isolate` to create a new stacking context, preventing child z-indexes from bleeding out and mixing with the rest of the page.

Object Fit/Position, Float & Clear

Object Fit & Position

Dictates how an `<img>` or `<video>` fits into its container.

object-cover: Fills container, cropping if needed (great for hero images)

object-contain: Scales without cropping

object-center / object-top: Aligns the image

```

```

Float & Clear

Mostly used for wrapping text around images. Modern layouts should use Flex or Grid instead.

float-left: Element floats to left

float-right: Element floats to right

clear-both: Forces element below floats

```
<article>
  
  <p>Text wraps around the image.</p>
  <div class="clear-both"></div>
</article>
```

Container

The container class sets the max-width of an element to match the min-width of the current breakpoint. **Note:** It does not center itself by default. You must add mx-auto.

```
<div class="container mx-auto px-4">
  Centered content with padding
</div>
```

Display Utilities

block: display: block;	inline-block: display: inline-block;
inline: display: inline;	flex: display: flex;
grid: display: grid;	hidden: display: none;

```
<!-- Hidden on mobile, flex on desktop -->
<div class="hidden md:flex">
  Menu Items
</div>
```

Box Sizing, Overflow, and More

Overflow

overflow-auto: Adds scrollbar if needed
overflow-hidden: Clips overflowing content
overflow-x-scroll: Always show horiz scrollbar

Visibility vs Display

invisible hides the element but keeps its space (visibility: hidden). **hidden** removes it from the layout flow entirely.

Aspect Ratio

Force an element to maintain a specific width-to-height ratio.

```
<!-- 16:9 Video Ratio -->
<iframe class="w-full aspect-video">
</iframe>
<!-- 1:1 Square Ratio -->

<!-- Custom Ratio -->
<div class="aspect-[4/3]"></div>
```

Mobile-First Breakpoints

Tailwind uses a **mobile-first** approach. Unprefixed utilities take effect on all screen sizes, while prefixed utilities (like `md:`) only take effect at that breakpoint and above.

`sm`: ≥ 640px (Large phones)
`md`: ≥ 768px (Tablets)
`lg`: ≥ 1024px (Laptops)
`xl`: ≥ 1280px (Desktops)
`2xl`: ≥ 1536px (Large Screens)

```
<!--
  w-full:    On mobile, take 100% width
  md:w-1/2:  On tablets & up, take 50% width
  lg:w-1/3:  On laptops & up, take 33% width
-->
<div class="w-full md:w-1/2 lg:w-1/3 bg-gray-200 p-4">
  Responsive Card
</div>
```

Dark Mode (dark:)

Use the `dark:` variant to style your site differently when dark mode is enabled (either via OS preference or manual class toggling).

```
<!-- Light mode: white bg, black text -->
<!-- Dark mode: black bg, white text -->
<div class="bg-white dark:bg-slate-900 text-black dark:text-white">
  <p>Hello Dark Mode!</p>
</div>
```

Arbitrary Values (JIT)

Need an exact value that isn't in your theme? Use square brackets `[]` to generate CSS on the fly without touching `tailwind.config.js`.

```
<!-- Custom exact width -->
<div class="w-[327px] h-[10vh]">...
</div>
<!-- Custom exact hex color -->
<p class="text-[#ff4499] bg-[#000000]">...</p>
<!-- Custom exact gap -->
<div class="grid gap-[2.75rem]">...
</div>
```

Utility Classes & Composition

What are Utilities?

Utilities are single-purpose CSS classes that map directly to a specific CSS property. For example, `flex` applies `display: flex;`, and `pt-4` applies `padding-top: 1rem;`.

Class Composition (Building a Button)

By combining multiple utilities on a single HTML element, you can create complex, custom components without writing a single line of custom CSS.

```
<!-- Combining layout, padding, font, color, and radius -->
<button class="flex items-center justify-center px-4 py-2 font-semibold text-white bg-blue-500 rounded-lg shadow-md">
  Click Me
</button>
```

State Variants (Pseudo-classes)

Hover, Focus, and Active States

Tailwind lets you conditionally apply utility classes in different states using modifiers like `hover:`, `focus:`, and `active:`. This completely replaces standard CSS pseudo-classes.

1. hover:

Triggers when the mouse is over the element.

```
<button class="bg-blue-500 hover:bg-blue-700 ...">
  Hover me
</button>
```

2. focus:

Triggers when the element receives keyboard focus.

```
<input class="border focus:border-blue-500 focus:ring-2 ...">
  <!-- Focus ring -->
</input>
```

3. active:

Triggers when the element is being clicked.

```
<button class="bg-blue-500 active:scale-95 ...">
  Click me (Shrinks)
</button>
```

Pro Tip: You can stack these variants! Example: `hover:focus:bg-red-500` applies the red background only when the element is both hovered AND focused.

Inside tailwind.config.js

The config file is the heart of your Tailwind setup. Use it to define your design system, custom colors, fonts, and plugins.

- **content:** Paths to all your HTML/JS files.
- **theme.extend:** ADD to the default theme.
- **theme (direct):** OVERRIDE the default theme.
- **plugins:** Add official/custom plugins (like forms, typography).

```
module.exports = {
  content: ["/src/**/*.html", "/src/**/*.jsx"],
  theme: {
    extend: {
      colors: {
        brand: "#0ea5e9", // Custom color
      },
      fontFamily: {
        sans: ['Inter', 'sans-serif'],
      },
    },
  },
  plugins: [
    require("@tailwindcss/forms"),
  ],
}
```

Recommended Architecture

While Tailwind doesn't enforce a folder structure, keeping your global CSS and config files organized is a best practice.

```
$ my-project/
$ | public/
$ | src/
$ | | components/      # Reusable UI
$ | | styles/
$ | | | index.css      # @tailwind directives
$ | | App.jsx
$ | | main.jsx
$ | tailwind.config.js  # Tailwind settings
$ | postcss.config.js  # PostCSS config
$ | package.json
```

Vite + React Installation

1. Create Project & Install Tailwind

```
$ npm create vite@latest my-project --  
--template react  
$ cd my-project  
$ npm install -D tailwindcss postcss  
autoprefixer  
$ npx tailwindcss init -p
```

The `-p` flag creates `postcss.config.js` which is required by Vite.

2. Configure `tailwind.config.js`

```
export default {  
  content: [  
    "./index.html",  
    "./src/**/*..{js,ts,jsx,tsx}",  
  ],  
  theme: { extend: {} },  
  plugins: [],  
}
```

Next.js App Router Setup

Modern Next.js makes setting up Tailwind incredibly easy. It's included by default when you run the create command.

```
$ npx create-next-app@latest my-app
```

During setup, simply answer **"Yes"** to the prompt:
Would you like to use Tailwind CSS? > No / Yes

Tailwind vs Bootstrap

- BS:** Gives pre-designed components (e.g., `.btn`). Sites often look the same.
- TW:** Gives raw utilities. You build the components. Sites look completely custom.
- BS:** Large file size if you override default styles heavily.
- TW:** Tiny file size because unused classes are purged at build time.

Tailwind vs Traditional CSS

- CSS:** You waste time inventing class names (e.g., `.sidebar-outer-wrapper`).
- TW:** No naming needed. Just use `flex p-4 bg-gray-100`.
- CSS:** Global scope issues. Changes in one file might break another page.
- TW:** Styles are localized to the HTML element. Deleting an element safely deletes its CSS.

Standard Tailwind CLI Setup

1. Install Tailwind via npm

```
$ npm install -D tailwindcss
$ npx tailwindcss init
```

3. Add Directives to CSS (src/input.css)

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

2. Configure template paths (tailwind.config.js)

Tell Tailwind exactly which files contain your class names so it can purge unused CSS.

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  // Scan these files for Tailwind classes
  content: ["../src/**/*.html,js,jsx,ts,tsx"],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

4. Start the CLI build process

```
$ npx tailwindcss -i ./src/input.css -o
./dist/output.css --watch
```

What is Tailwind CSS?

Tailwind CSS is a highly customizable, low-level **utility-first CSS framework**. Unlike semantic frameworks (like Bootstrap), it doesn't give you pre-built components (like cards or buttons). Instead, it gives you building blocks (utility classes) to build any design directly in your HTML.

Advantages & Features

- **No Context Switching:** You style elements without leaving your HTML/JSX file.
- **Tiny Production Build:** Purges unused CSS automatically (often <10kb).
- **Highly Customizable:** Everything can be tailored in `tailwind.config.js`.
- **Responsive out-of-the-box:** Easy mobile-first media queries.

Traditional CSS vs Utility Classes

Instead of inventing class names like `.chat-notification` and writing custom CSS for it, you apply predefined utility classes directly to the markup. This eliminates the hardest part of CSS: naming things and managing file sizes.

TRADITIONAL WAY

```
<!-- You have to write custom CSS for this -->
<div class="chat-notification">
  <div class="chat-logo-wrapper">
    
  </div>
  <div class="chat-content">
    <h4 class="chat-
title">ChitChat</h4>
    <p class="chat-message">You have a
new message!</p>
  </div>
</div>
```

TAILWIND WAY

```
<!-- Styles applied directly via utilities -->
<div class="p-6 max-w-sm mx-auto bg-
white rounded-xl shadow-lg flex items-
center space-x-4">
  <div class="shrink-0">
    
  </div>
  <div>
    <div class="text-xl font-medium
text-black">ChitChat</div>
    <p class="text-slate-500">You have
a new message!</p>
  </div>
</div>
```

PART 1: SETUP & FUNDAMENTALS

- 01. Introduction & Comparisons
- 02. Project Setup (Vite, Next.js)
- 03. Tailwind Fundamentals & Arbitrary Vals

PART 2: CORE LAYOUT

- 04. Layout (Display, Position, Z-Index)
- 05. Flexbox Architecture
- 06. CSS Grid Layout
- 07. Spacing (Margin, Padding, Sizing)

PART 3: TYPOGRAPHY & AESTHETICS

- 08. Typography (Fonts, Text Align)
- 09. Backgrounds & Gradients
- 10. Borders, Radius & Rings
- 11. Effects (Shadows, Opacity, Filters)

PART 4: MEDIA & MOTION

- 12. Tables (Layout, Spacing)
- 13. SVG (Fill, Stroke)
- 14. Transitions (Duration, Delay)
- 15. Transforms (Scale, Rotate)
- 16. Animations & Keyframes
- 17. Interactivity (Cursor, Select)

PART 5: STATES & ARCHITECTURE

- 18. Responsive Design (Breakpoints)
- 19. State Variants (hover, focus, peer)
- 20. Dark Mode Strategies
- 21. Customization (tailwind.config)
- 22. Directives (@apply, @layer)

PART 6: ECOSYSTEM & REAL WORLD

- 23. Performance & Tree Shaking
- 24. Tailwind with Frameworks
- 25. UI Components (Cards, Navbars)
- 26. Best Practices & Clean Code
- 27. Real Projects (Dashboard, SaaS)

✦ A complete 27-chapter reference guide to master Tailwind CSS styling & architecture!